

Wells Fargo Preparation

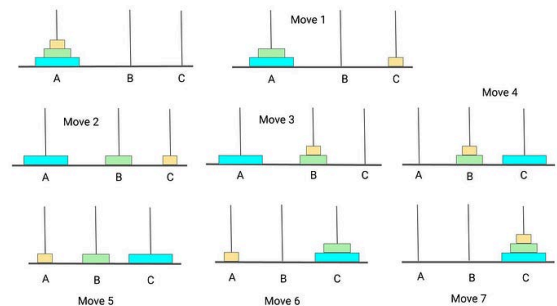


24 LPA(16+2+2+retention)...

3 Coding Question(2.5 question) (2hr)

- 1 leetcode easy (classic tower of hanoi..)
- 1 leetcode medium
- 1 leetcode hard

MCQs → Aptitude, CS/Verbal, Business



Technical Interview:

- Write some code to implement checks on browser side(related to my project)
- Give an example of where a **private constructor** is used.
- Explain **inheritance** in c++
- Can **destructors** be declared private? why/why not?
- What are views in the database? Their advantages vs disadvantages
- Give an example where the use of views is discouraged?
- Triplet sum (gfg)

More questions over here - [link](#)

Our collage Senior experience - [link](#)

Get in the practice of OA: [link](#)

Other's experience: [link](#)

DBMS resource: [link](#)

OA question: [link](#)

OS resource: [link](#)

Tricky OOPs in c++: [link](#)

All questions: [link](#)

Based on multiple candidate experiences and online resources, here's a summary of the previous year's Wells Fargo Online Assessment (OA) format for Intern Analyst and similar roles:

Typical OA Format

1. Section 1: English/Verbal Reasoning

- **Content:** Questions on grammar, comprehension, paraphrasing, and error identification.
- **Format:** Often around 10–15 multiple-choice questions.
- **Time Limit:** Approximately 10–15 minutes.
- **Purpose:** Tests your language skills and your ability to quickly interpret written information.

(Sources: [geeksforgeeks.org](https://www.geeksforgeeks.org), [medium.com](https://www.medium.com))

2. Section 2: Business Aptitude/Logical/Numerical Reasoning

- **Content:** Questions covering basic arithmetic, percentages, data interpretation, logical puzzles, and sometimes business-related concepts.
- **Format:** Typically 7–14 MCQs.
- **Time Limit:** Around 20–25 minutes.
- **Purpose:** Assesses your numerical and logical reasoning skills under time pressure.

(Sources: [geeksforgeeks.org](https://www.geeksforgeeks.org), [mconsultingprep.com](https://www.mconsultingprep.com))

3. Section 3: Coding Assessment

- **Content:** Coding problems that may involve algorithms, data structures, and problem-solving challenges.
- **Format:** Usually 2–4 coding questions to be solved on an online coding platform (similar to HackerRank).
- **Time Limit:** Generally, around 40–70 minutes for the coding portion (some candidates report 2 questions in about 70 minutes).
- **Purpose:** Evaluates your programming skills and your ability to write clean, efficient code under timed conditions.

(Sources: [geeksforgeeks.org](https://www.geeksforgeeks.org), [reddit.com](https://www.reddit.com), [glassdoor.com](https://www.glassdoor.com))

Question 1: The city of Hackerland can be represented as a two-dimensional grid of size $n \times m$. Each cell is either empty (a dot character), has an obstacle (an asterisk *), an S or an E that represent the start and end points respectively.

One can move in four directions: up, down, left, and right. The goal is to move from the starting point to the ending point such that in the path, the minimum distance from any obstacle is as large as possible. Return this minimum possible distance.

The distance between any two points on the grid with coordinates (r_1, c_1) and (r_2, c_2) is calculated as $|r_1 - r_2| + |c_1 - c_2|$, where $|a|$ is the absolute value of integer a .

Notes:

- One can visit a cell with an obstacle if necessary, Le. no other path exists.
- A cell can be visited only once.

Example: Consider the following grid of size 4*4:

S	.	.	*
.	.	*	.
.	.	.	.
.	.	.	E

The optimal path is shown in the 'Cell' column. Coordinates of any of the nearest obstacles is in 'Obstacle'.

Cell	Nearest Obstacle	Distance
(0, 0)	(0, 3)*	3
(1,0)	(1,2)	2
(2,0)	(1,2)	3
(3,0)	(1,2)	4
(3,1)	(1,2)	3
(3,2)	(1,2)	2
(3,3)	(0,3)*	3

The blocked cell (1, 2) is also 3 units distant.

Return 2, the closest distance to any obstacle along the path.

Function Description

Complete the function findMaximumDistance in the editor below.

findMaximumDistance has the following parameter: string grid[n]: An array of strings that represent the rows of the grid.

Returns int the largest possible minimum distance from an obstacle in any path from the starting point to the ending point.

Constraints

- $2 \leq n, m \leq 200$
- `grid[i][j] = '.'` if the cell is empty

Question 2 A permutation of n numbers is a sequence where each number from 1 to n appears exactly once. For a given permutation p and any arbitrary array arr , a permutation operation is defined as

- For each index i ($1 \leq i \leq n$) $temp_arr[i] = arr[p[i]]$
- $arr = temp_arr$

Given a permutation p of n numbers, start with any arbitrary array arr of n distinct elements and find out the minimum number of permutation operations (at least 1) needed in order to reach the original array. Since the answer can be quite large, return the answer modulo (10^9+7) .

Example $P=[1,3,2]$ In the above example, $n=3$. Taking any arbitrary array $arr = [7, 8, 9]$

- In each operation:
 - the element at index 1 stays at index 1
 - the element at index 2 gets mapped to index 3
 - the element at index 3 gets mapped to index 2
- After applying operation for the first time on arr , the resulting array is $[7, 9, 8]$.
- After applying operation for the second time the resulting array is $[7, 8, 9]$.

After 2 operations we get back to the original array, hence we return 2 as the answer.

Function Description Complete the function `countOperations` in the editor below.

`countOperations` has the following parameter:

`int p[n]`: a permutation of the integers from 1 to n

Returns `int`: the number of operations required modulo (10^9+7)

Constraints 1. $1 \leq n \leq 10^5$ 2. $1 \leq p[i] \leq n$

3. contains all distinct elements, the integers 1 through n Input

```
#include <bits/stdc++.h>
using namespace std;

long long lcm(long long a, long long b) {
    return (a / __gcd(a, b)) * b;
}

int countOperations(vector<int>& p) {
    const int MOD = 1e9 + 7;
    int n = p.size();
    vector<int> inv_p(n + 1, 0); // 1-based indexing

    // Compute the inverse permutation
    for (int i = 0; i < n; ++i) {
        int x = p[i];
        inv_p[x] = i + 1; // adjust to 1-based indexing
    }

    vector<bool> visited(n + 1, false);
    long long result = 1;

    // Detect cycles and compute LCM of cycle lengths
    for (int x = 1; x <= n; ++x) {
        if (!visited[x]) {
            int cycle_length = 0;
            int current = x;
            while (!visited[current]) {
                visited[current] = true;
                cycle_length++;
                current = inv_p[current];
            }
            // Update result with LCM of current cycle length
            result = lcm(result, cycle_length);
        }
    }

    return result % MOD;
}
```

Question 3 There is a string input_str consisting of characters '0' and '1' only and an integer k. Find a substring of string input_str such that:

- The number of '1's is equal to k
- It has the smallest length
- It is lexicographically smallest

Note: It is guaranteed that the answer always exists.

Example input str= "0101101" k=3 Some of the possible substrings following the first condition: "01011" "1101" "1011"

The substring that is smallest in length and lexicographically smallest is "1011". It can be proven that there is no other substring that is smaller than "1011" in length and lexicographic order. Hence the answer is "1011"

Function Description Complete the function getSubstring in the editor below.

getSubstring has the following parameters:

- string input_str: a string that consists of '0' and '1'
- int k: the number of '1's in the answer

Returns string: the substring that meets the given conditions

Constraints $1 \leq k \leq \text{length of } s \leq 10^3$

$s[i]$ in the set ('0', '1')

The number of '1' characters in string input_str is always greater than or equal to k.

```
#include <bits/stdc++.h>
using namespace std;

string getSubstring(string input_str, int k) {
    /* generate all possible valid substring */
    int n = input_str.size();
    string best = "";
    bool found = false;

    for(int i = 0; i < n; i++){
        int count{};
        for(int j = i; j < n; j++){
            if(input_str[j] == '1') count++;
            if(count == k){
                string candidate = input_str.substr(i, j-i+1);
                if(!found || candidate.size() < best.size() ||
                    (candidate.size() == best.size() and candidate < best)){
                    best = candidate;
                    found = true;
                }
                break;
            }
        }
    }
    return best;
}

int main() {
    cout << getSubstring("0101101",3) << endl;
    return 0;
}
```

The interview had 3 rounds in total (1 OA, 2 Interviews) :

R1 : The OA had 3 sections here :

Section 1: 12 English grammar and comprehension questions to be done in 15minutes.

Section 2: 14 Business Analytics and Data Interpretation problems in 25 minutes.

Section 3: 2 Coding questions to be done in 1 hour. I was asked 1 medium and 1 easy problem in the test. However, everyone had different sets of problems in this round.

Problems :

1: Given an array of n elements, print the distance between the largest inversion pair in the array. (Inversion Pair: For $i < j$, $A[i] > A[j]$)

2: Given an array of n elements, return another array whose elements are the digit sum of the elements of the original array.

R2 : After a warm welcome, I was asked to give a brief introduction, along with the which Programming languages I'm aware of. Looking at my EE background, he asked me if I would be able to answer core CS at all, to which I gave the go ahead. He asked me the fundamental pillars of OOPs, their applications in real life, compilers vs interpreters, processes and threads, and some Java questions.

I was given 2 coding challenges which I explained and coded too:

1: Given 2 strings, find the length of the longest common substring of the 2 strings.

2: Given an array representing stock prices, find the best Profit one can obtain by buying a stock on one day and selling it on a later day.

Question 1 could be optimised using BIT/Merge sort algo

```

// Function to find the maximum inversion pair distance
int maxInversionDistance(vector<int>& arr) {
    int n = arr.size();
    int maxDist = 0;

    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            if (arr[i] > arr[j]) {
                maxDist = max(maxDist, j - i);
            }
        }
    }
    return maxDist;
}

// Function to compute digit sum of a number
int digitSum(int num) {
    int sum = 0;
    while (num > 0) {
        sum += num % 10;
        num /= 10;
    }
    return sum;
}

// Function to transform array into digit sum array
vector<int> digitSumArray(vector<int>& arr) {
    vector<int> result;
    for (int num : arr) {
        result.push_back(digitSum(num));
    }
    return result;
}

```

```

struct InversionInfo {
    int value;
    int index;
};

// Merge function to count inversions
int mergeAndCount(vector<InversionInfo>& arr, int left, int mid, int right,
int& maxDist) {
    int i = left, j = mid + 1;
    vector<InversionInfo> temp;
    int invCount = 0;

    while (i <= mid && j <= right) {
        if (arr[i].value <= arr[j].value) {
            temp.push_back(arr[i++]);
        } else {
            invCount += (mid - i + 1);
            maxDist = max(maxDist, arr[j].index - arr[i].index); // Update max
distance
            temp.push_back(arr[j++]);
        }
    }

    while (i <= mid) temp.push_back(arr[i++]);
    while (j <= right) temp.push_back(arr[j++]);

    for (int k = left; k <= right; k++)
        arr[k] = temp[k - left];

    return invCount;
}

```



```

// Merge Sort function
int mergeSortAndCount(vector<InversionInfo>& arr, int left, int right, int&
maxDist) {
    if (left >= right) return 0;

    int mid = left + (right - left) / 2;
    int invCount = 0;

    invCount += mergeSortAndCount(arr, left, mid, maxDist);
    invCount += mergeSortAndCount(arr, mid + 1, right, maxDist);
    invCount += mergeAndCount(arr, left, mid, right, maxDist);

    return invCount;
}

```

```

// Function to find the maximum inversion distance
int maxInversionDistance(vector<int>& arr) {
    int n = arr.size();
    vector<InversionInfo> indexedArr;
    for (int i = 0; i < n; i++)
        indexedArr.push_back({arr[i], i});

    int maxDist = 0;
    mergeSortAndCount(indexedArr, 0, n - 1, maxDist);
    return maxDist;
}

```

```

int main(){
    string s1, s2;
    cin >> s1 >> s2;
    int n = s1.length();
    int m = s2.length();
    vector<vector<int>> dp(n, vector<int>(m, 0));
    int length = 0;

    // Initialize the first row.
    for (int j = 0; j < m; j++){
        if(s1[0] == s2[j]){
            dp[0][j] = 1;
            length = 1;
        }
    }

    // Initialize the first column.
    for (int i = 0; i < n; i++){
        if(s1[i] == s2[0]){
            dp[i][0] = 1;
            length = 1;
        }
    }

    // Fill the rest of the DP table.
    for (int i = 1; i < n; i++){
        for (int j = 1; j < m; j++){
            if(s1[i] == s2[j]){
                dp[i][j] = dp[i-1][j-1] + 1;
                length = max(length, dp[i][j]);
            }
            // Else, dp[i][j] remains 0 (no common substring ending here)
        }
    }

    cout << length << "\n";
    return 0;
}

```

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```

int main() {
    int n;
    cin >> n; // number of days (stocks)
    vector<int> prices(n);
    for (int i = 0; i < n; i++) {
        cin >> prices[i];
    }

    int minPrice = INT_MAX, maxProfit = 0;
    for (int price : prices) {
        minPrice = min(minPrice, price);
        maxProfit = max(maxProfit, price - minPrice);
    }

    cout << maxProfit << "\n";
    return 0;
}

```

OOPs(Object Oriented Programming)..... Is a Programming paradigm

Class is user defined data-type. Instance of this class is called Object. Every member of a class in c++ is Private by default. We have 3 access modifiers: public,private,protected.

Constructors: A special type of method that is invoked each time an object of a class is created. Default constructors <— Generated by compiler

4 Pillars of OOPs:

- Encapsulation: Bundling the data and methods that work on that data. How to extract encapsulated properties of a class from another class? **Getters and setters**
- Abstraction: Hiding complex things behind a procedure that makes it looks easy.
- Inheritance: child(**derived**) class will inherit all members of parent(**base**) class.
- Polymorphism: Ability of an object or a method to have many forms.

```
#include <bits/stdc++.h>
using namespace std;
```

```
/* Abstract class: whoever wants to sign this contract of Abstract employee needs to give
implementation of this class too. */
```

```
class AbstractEmployee{    /* Pure virtual function */
    virtual void AskForPromotion() = 0;
};
```

```
class Employee:AbstractEmployee {
private:
    // members of this class
    string Company; int Age;
protected:
    string Name;          /* can be accessed in derived class */
public:
    void setName(string name){ /* Getter */
        Name = name;
    }
    string getName(){ /* Setter */
        return Name;
    }
    void setAge(int age) {
        if(age >= 18) Age = age; /* used for setting validation rules */
    }
    // behaviour of this class
    void introduce(){ cout << "hello my friend\n"; }
    // constructor
    Employee(string name,string company,int age){
        Name = name;
        Company = company;
        Age = age;
    }
    // i am overriding
    void AskForPromotion(){
```

```

        if(Age > 30) cout << Name << "got promoted" << "\n";
        else cout << "Sorry, no promotion." << "\n";
    }
    virtual void Work(){
        cout << Name << " is checking emails, task backlog, performing those task.. "
<< endl;
    }
};

```

```

class Developer:public Employee{ /* for public inheritance */
public:
    string favProgrammingLanguage;
    Developer(string name,string company,int age,string language)
        : Employee(name,company,age);
    {
        favProgrammingLanguage = language;
    }
    void fixBug(){
        cout << getName() << " fixed bug using " << favProgrammingLanguage <<
"\n";
    }
    void Work(){
        cout << Name << " is writing code in " << favProgrammingLanguage << endl;
    }
};

```

```

class Teacher:public Employee {
public:
    string Subject;
    void PrepareLesson(){
        cout << Name << " is preparing " << Subject << " lesson " << endl;
    }
    Teacher(string name,string company,int age,string subject)
        :Employee(string name,string company,int age)
    {
        Subject = subject;
    }
    void Work(){
        cout << Name << " is teaching " << Subject << endl;
    }
};

```

```

int main(){
    /* how to access the members of this class
    Employee employee1;
    employee1.Name = "Ramesh";
    employee1.Company = "WF";
    employee1.Age = 20;
    employee1.introduce();

```

```

*/
Employee employee1 = Employee("Ramesh","WF",20);
employee1.introduce();
Employee employee2 = Employee("Swastika","WF",20);
employee2.introduce();

Developer d = Developer("Swastika","WF",20,"C++");
d.fixBug();
d.AskForPromotion();

Teacher t = Teacher("Jack","Cool School",35,"History");
t.PrepareLesson();
t.AskForPromotion();

d.Work();
t.Work();

Employee* e1 = &d;
Employee* e1 = &d;
e1->Work(); e2->Work();
employee2.setAge(26);
cout << "Employee name:" << employee2.getName() << " is " << employee2.getAge()
<< " years old " << endl;

employee1.AskForPromotion();
employee2.AskForpromotion();
cin.get();      /* acts like a break point */
}

```

Static/Early(Compile Time) and Dynamic/Late(Running Time) Binding in C++

Static binding happens by **default** by operator overloading or function overloading(same function name with different parameters).

Dynamic binding happens when we use virtual function or function overriding.

Extra Topics to study from here:- [studyLink](#)

Cheat Sheet for Aptitude Type Questions

Type of question	Tricks
1. Number Series	- Look at Differences: Check consecutive differences or second-level differences. - Even/Odd Position Patterns: Compare terms at even indices vs. odd indices. - Ratios / Multiples: See if each term is multiplied or divided by a fixed factor. - Special Sequences: Look for squares, cubes, primes, factorials, or arithmetic/geometric progressions.
2. Missing Number / Next Shape (Non-verbal Reasoning)	- Count Elements / Sides: For shapes, track number of sides, corners, or line segments. - Transformations: Look for rotation, reflection, or shifting patterns. - Color / Shading Changes: Note if shading or color toggles in a sequence. - Position or Orientation: Shapes may move from left to right, up/down, or flip.
3. Data Interpretation	- Read Carefully: Identify units (e.g., thousands vs. millions) and check axis labels. - Approximation vs. Exact: Decide if approximate calculations suffice. - Compare Correctly: Convert all data to the same scale or unit. - Common Graphs: Bar, pie, line charts—focus on changes or percentage splits.
4. Percentages	- Basic Formula: $(\text{Change} / \text{Original}) \times 100$. - Successive Changes: For back-to-back percentage increases/decreases, use compound multipliers (e.g., +20% then -20% is $1.2 \times 0.8 = 0.96$, a net 4% decrease). - Percentage Points vs. % Change: Distinguish between absolute difference and relative percentage.
5. Ratio & Proportion	- Cross-Multiplication: $a/b = c/d \Rightarrow ad = bc$. - Part Splitting: If the ratio is 2:3, total parts = 5. Distribute amounts accordingly. - Scaling: Ratios scale up or down consistently (e.g., 2:3 is the same as 4:6).
6. Time & Work	- Work = Rate $\times$ Time: Or rate = $1/\text{time}$ for each person. - Combined Work: Sum individual rates for total rate. - Partial Work: Track fractions of work done if tasks switch mid-way.
7. Time, Speed & Distance	- Formula: Distance = Speed $\times$ Time. - Relative Speed: If objects move in opposite directions, add speeds; if same direction, subtract. - Unit Conversions: Keep track of km/h to m/s or mph to fps.
8. Simple & Compound Interest	- Simple Interest (SI): $SI = (P \times R \times T) / 100$. - Compound Interest (CI): $A = P (1 + R/100)^T$; $CI = A - P$. - Effective Rate: For multiple compounding periods, adjust the rate or time accordingly.
9. Profit & Loss	- Cost Price (CP), Selling Price (SP), Profit/Loss: Profit or Loss = $SP - CP$. - Profit/Loss %: $(\text{Profit or Loss} / CP) \times 100$. - Mark-up & Discount: Combine percentage increases and decreases carefully.
10. Permutations & Combinations	- Permutation (nPr): $n! / (n-r)!$. Order matters. - Combination (nCr): $n! / (r!(n-r)!)$. Order does not matter. - “At Least / At Most”: Often requires summing multiple combination cases or using complementary counting.

11. Probability	- Basic Formula: Probability = (Favorable outcomes) / (Total possible outcomes). - Independent Events: $P(A \text{ and } B) = P(A) \times P(B)$. - Dependent Events: Adjust probabilities after each selection (without replacement).
12. Logarithms	- Key Properties: $\log(a \times b) = \log(a) + \log(b)$; $\log(a^n) = n \log(a)$. - Base Changes: $\log_a b = (\log b) / (\log a)$. - Common vs. Natural Logs: log base 10 vs. ln base e.
13. Mixtures & Allegations	- Difference Method: For mixing two items with different concentrations, use the "cross" method to find the ratio. - Weighted Averages: Weighted average = (Sum of products) / (Sum of weights).
14. Ages / Partnership	- Ages: Form linear equations from relationships (e.g., "A is twice B's age in 5 years" $\Rightarrow A + 5 = 2(B + 5)$). - Partnership: Profit is split by (Investment $\times$ Time).
15. Coding-Decoding (Verbal Reasoning)	- Letter Shifts: Map letters by shifting forward/backward in the alphabet. - Pattern Recognition: Check positions of letters (A=1, B=2, etc.). - Substitution: Some codes replace letters with symbols or numbers.
16. Blood Relations	- Reference Person: Pick "you" or a known family member as a reference. - Family Tree: Draw a quick diagram to keep track of relationships. - Terminology: "Maternal uncle," "paternal aunt," etc., to avoid confusion.
17. Direction & Distance	- Track Movements: Use a rough sketch for left, right, north, south turns. - Angle Changes: 90° for right angles, 180° for about-turn. - Final Position: Compare final coordinates or direction from the start.
18. Seating Arrangement	- Linear vs. Circular: In circular arrangements, fix one person's seat and arrange the rest relative to them. - Left / Right: Carefully note perspective (the subject's left vs. your viewpoint). - Clues: Translate each statement into a partial arrangement diagram.
19. Syllogisms	- Venn Diagrams: Represent each statement with sets and see overlaps. - Universal & Particular: All, No, Some, Some not. - Check Valid Conclusions: Does the conclusion logically follow from the premises?
20. Clocks & Calendars	- Clock Angles: Minute hand moves 6° per minute, hour hand 0.5° per minute. - Calendar: Use day offsets, or known reference days, to find day of the week for given dates.

Wells Fargo OA Sample Test (2 Hours)

Total Duration: 120 minutes

Instructions:

- Read each question carefully.
 - Answer all multiple-choice questions in Section A.
 - Write clean, well-commented code in Section B.
 - Provide a structured response for the open-ended question in Section C.
 - Manage your time: approximately 40 minutes for Section A, 70 minutes for Section B, and 10 minutes for Section C.
-

Section A: Aptitude, Verbal, Logical & Numerical Reasoning (40 Minutes)

Question 1: Verbal Synonym

Which word is the best synonym for "succinct" in the sentence below?

"The speaker delivered a succinct presentation that held everyone's attention."

- A. Verbose B. Concise C. Diffuse D. Ambiguous
-

Question 2: Sentence Correction

Identify the error in the following sentence:

"Each of the applicants have been informed about the interview schedule."

- A. "Each" should be replaced with "All" B. "have been" should be "has been"
C. "about" should be "on" D. There is no error
-

Question 3: Reading Comprehension

Read the passage below and answer the question:

"Global markets have shown signs of recovery after a prolonged period of stagnation. Analysts attribute this improvement to a combination of fiscal stimulus measures and the gradual reopening of economies."

What is the primary reason for the market recovery according to the passage?

- A. Reopening of economies B. Fiscal stimulus measures
C. A combination of both D. None of the above
-

Question 4: Logical Reasoning

Find the next number in the series: 2, 6, 12, 20, ?

- A. 28 B. 30 C. 32 D. 36
-

Question 5: Numerical Reasoning

A product's price is increased by 20% and then decreased by 20%. What is the net effect on the price?

- A. No change B. 4% increase C. 4% decrease D. 20% decrease
-

Question 6: Data Interpretation

A company's revenue grows from \$1.2 million to \$1.5 million in one year. What is the percentage increase?

- A. 20% B. 25% C. 30% D. 35%
-

Question 7: Verbal Analogies

Book is to Reading as Fork is to:

- A. Drawing B. Writing C. Stirring D. Eating
-

Question 8: Critical Reasoning

Which of the following best summarizes the conclusion of the argument:

"Regular exercise improves health because it increases cardiovascular strength, reduces stress, and promotes mental well-being."

- A. Regular exercise is unnecessary for good health. B. Regular exercise improves health.
C. Exercise only benefits the heart. D. None of the above.
-

Question 9: Numerical Problem

If 3 apples cost \$1.80, how much will 7 apples cost at the same rate?

- A. \$4.20 B. \$4.50 C. \$3.80 D. \$5.00
-

Question 10: Logical Deduction

"All A are B. No B are C. Which of the following must be true?"

- A. Some A are not C B. No A are C C. All C are A D. Some C are A
-

Section B: Coding Challenges (70 Minutes)

Problem 1: Reverse a String

Write a function that takes a string as input and returns the string reversed.

- **Example:**
 - **Input:** "WellsFargo"
 - **Output:** "ograFsllEw"
-

Problem 2: Two Sum

Given an array of integers and a target integer, write a function that returns the indices of the two numbers that add up to the target. Assume exactly one solution exists, and you cannot use the same element twice.

- **Example:**
 - **Input:** nums = [2, 7, 11, 15], target = 9
 - **Output:** [0, 1]
-

Problem 3: Count Vowels

Write a function that takes a string as input and returns the count of vowels (a, e, i, o, u, case-insensitive) in the string.

- **Example:**
 - **Input:** "WellsFargo"
 - **Output:** 3
(Explanation: "e", "a", and "o" are vowels.)

Section C: Analytical/Open-Ended Question (10 Minutes)

Analytical Question:

Imagine you are tasked with analyzing a dataset of customer transactions at Wells Fargo to identify potentially fraudulent activities. Describe your approach step-by-step, including:

- **Data Preprocessing:** How would you clean and prepare the data?
- **Feature Engineering:** What features would you derive to flag anomalies?
- **Detection Techniques:** Which statistical methods or machine learning models might you use?
- **Handling False Positives:** How would you manage the balance between detecting fraud and avoiding false alarms?

Provide a structured, concise answer using bullet points or numbered steps.

Below is a set of interview questions tailored to your resume, assuming I'm an interviewer at Wells Fargo evaluating you for a technical intern or analyst position. They span technical depth, project insights, and your overall fit for the role and company.

1. Introductory & Motivational

1. **Tell me about yourself** and walk me through your background. What led you to pursue Computer Science Engineering?

My name is Ramesh Chandara Soren and I'm pursuing my CS degree here. I originally belong from Odisha but have lived in Bengal for 13+ yrs.

To be honest i don't have a very huge self motivation to do CS. My motivation is my friends and family who wanted me to pursue CS.

2. **Why Wells Fargo?** What interests you about working at a financial institution, and how do you see your technical skills contributing here?

Wells Fargo is investing in emerging technologies to accelerate its digital transformation strategy. **AI, big data, cloud, and payments** are among the key technologies under focus for the company.

The annual ICT spending of Wells Fargo was estimated at \$4 billion in 2024.

Central to the digital infrastructure strategy is Wells Fargo's selection of two industry leaders as its public cloud providers: **Microsoft Azure** as its primary public cloud provider and Google Cloud providing additional business-critical public cloud services.

Wells Fargo has **rolled out AI capabilities in some branches to try to make branch-based bankers' and tellers' jobs easier** as they deal with "hundreds" of different policies and procedures in the course of their work

2. Questions About Your Internship Experience

[link](#)

1. **Flood Simulation Project:**

- Can you elaborate on the **web application** you're developing for flood simulation? What technologies are you using on the front end and back end?

Front-end: HTML,CSS,JS, we are avoiding React

Backend: Expressjs, Nodesjs, Openweather API....

- How do you handle the **3D modeling** aspects? Are you using any particular libraries or frameworks to render or simulate 3D environments?

This is still an idea, we haven't implemented any parts of it. Though we have gathered some ideas from the internet, we are very far from yet learning them and implementing them on our project. Some of the ideas are ArcGIS Pro, Hydraulic and Hydrodynamic Models....

- What are some **challenges** you faced—performance, accuracy of the simulation, or something else—and how did you address them?

Finding a dataset for training our AI models took weeks of surfing the net and government sites and asking local people about it.

Deploying Weather API using vercel was also very difficult. Setting up the environment and following each step carefully was difficult.

2. Collaboration & Research:

- Since it's a Research and Development internship, how did you **collaborate** with faculty or peers?

I have consulted two professors of our department, to help us out during the project time. Discussing ways on what new features to bring and how to implement that within a reasonable time limit that we had.

- Did you apply any **data-driven approaches** or real-world data to validate your flood models?

We have used a Real dataset for training our flood model. Though it has a good Accuracy, we haven't tested it live yet.

3. Real-Time Chess Application

1. Architecture & Tech Stack:

- You mentioned using **React, Node.js, and WebSocket**. How do you **manage real-time updates** and maintain game state consistency among multiple players?
- Can you explain how you **integrated chess.js** for rule enforcement? Did you encounter any edge cases (e.g., special moves like castling, en passant)?

2. Scalability & Error Handling:

- How would you **scale** this application if thousands of concurrent games are happening?
- What happens when a player **disconnects**? How do you handle timeouts, illegal moves, or reconnection logic?

4. Gemini Chatbot

1. Core Functionality & Stack:

- You built a chatbot with **Python (Flask) + React**. Can you walk me through the **API design** between your Flask backend and React frontend?

Backend (Flask):

- **RESTful Endpoints:** The Flask backend exposes a set of RESTful endpoints (e.g., /chat, /authenticate, /status) to handle requests.
- **JSON Communication:** All data is exchanged in JSON format. The backend receives user input from the React app, processes it (e.g., by forwarding it to the Gemini Wrapper), and then returns the generated response.
- **Error Handling & Security:** Flask includes middleware to handle errors gracefully and employs token-based (e.g., JWT) authentication to secure API calls.

Frontend (React):

- **Asynchronous Communication:** The React frontend uses libraries such as Axios or the Fetch API to send asynchronous HTTP requests to the Flask API.
- **State Management:** Responses from the backend update the UI state, ensuring the conversation flow is smooth and dynamic.
- **Security Considerations:** React works with secure headers and tokens provided during login, ensuring that only authenticated sessions can access sensitive endpoints.

- What is the **Gemini Wrapper** you referenced, and how does it simplify interactions with large language models or other AI services?

The **Gemini Wrapper** is essentially an abstraction layer that simplifies the integration with large language models or AI services. Its main benefits include:

- **Simplified API Calls:** Instead of handling raw API requests and responses to an external AI service, the wrapper provides clean, high-level functions. This lets you focus on building chatbot logic rather than managing low-level communication details.
- **Consistency:** It standardizes inputs and outputs, meaning you can swap out underlying AI providers with minimal changes to your codebase.
- **Error & Context Management:** The wrapper manages context persistence across turns and implements built-in error handling, which is critical for maintaining conversational state in a chatbot environment.

2. Use Cases & Security:

- Since it's for **medical use** and practitioners, how do you handle **sensitive data** or ensure user privacy?

Since the application is designed for medical practitioners and might handle sensitive data, the following measures are critical:

• Data Encryption:

- **In Transit:** All communications between the React frontend and Flask backend occur over HTTPS/TLS to protect data from interception.
- **At Rest:** Sensitive data is stored in encrypted databases, ensuring that even if a breach occurs, the data remains unreadable.

• Compliance & Access Controls:

- The system is designed to comply with healthcare regulations (e.g., HIPAA) by enforcing strict data access controls.
- Role-based access ensures that only authorized personnel can view or manipulate sensitive data.

• Anonymization & Minimal Data Storage:

3. Where possible, patient data is anonymized, and only the essential information needed for the conversation is retained.
4. Audit logs and sensitive details are kept to a minimum and for only as long as necessary for legal or operational purposes.

- What would be your approach to **logging** or **monitoring** chat interactions?

For a production system—especially one handling sensitive medical data—robust logging and monitoring are essential:

• Centralized Logging:

- Implement a logging solution (using tools like the ELK Stack or cloud-based alternatives) to capture and centralize logs from both the Flask backend and the React frontend.
- Logs include request details, API errors, and system events.

• Data Redaction:

- Ensure that any logs that might contain sensitive information are either anonymized or redacted before being stored. This helps maintain user privacy while still providing the necessary diagnostic data.

- **Real-time Monitoring & Alerts:**

- Set up real-time monitoring to track unusual patterns (e.g., sudden spikes in error rates or access anomalies) and configure alerts for potential security breaches.
- This can be integrated with a security information and event management (SIEM) system, enabling proactive incident response.

- **Compliance Audits:**

- Regular audits and reviews of log data help ensure that the system adheres to compliance standards and that any data breaches are identified and remediated promptly.
-

5. University Projects

5a. Breast Cancer Detection (4th SEM Minor Project)

1. Model & Techniques:

- You built a **Logistic Regression Model** with **Ridge/Lasso** regularization. Why did you choose logistic regression over other classifiers (e.g., SVM or random forest)?

I chose logistic regression primarily for its simplicity and interpretability. Its coefficients directly indicate the weight and direction of each predictor, which is critical in medical settings where understanding how each variable (like tumor size or patient age) influences the risk of breast cancer is essential. While SVMs or random forests can be powerful, they often act as “black boxes” with more complex decision boundaries. With regularization (Ridge/Lasso), logistic regression helps control overfitting and can perform comparably well on moderate-sized datasets while remaining transparent to clinicians.

- How did you **evaluate** your model's performance (beyond accuracy, precision, recall)? Why are these metrics important in a medical context?

Beyond accuracy, precision, and recall, I evaluated the model using:

- **AUC-ROC:** This measures the model's ability to distinguish between malignant and benign cases across all threshold levels. A high AUC indicates strong overall discrimination.
- **F1 Score:** Particularly useful when dealing with imbalanced classes, it harmonizes precision and recall to ensure a balanced performance.

- **Calibration Curves and Brier Score:** These assess whether the predicted probabilities align well with actual outcomes, which is vital for risk stratification in a clinical context.

These metrics are crucial in medical applications because they help quantify both the likelihood of missing a cancer case (false negatives) and the risk of unnecessary interventions due to false alarms (false positives).

2. Overfitting & Cross-Validation:

- You mentioned **k-fold cross-validation**. How did you pick the number of folds, and what insights did you gain from the validation results?

I applied k-fold cross-validation (using 10 folds) to obtain robust performance estimates while mitigating the risk of overfitting. Ten folds offered a good balance by providing sufficient training data in each iteration and reducing the variance in the performance estimates. This approach helped reveal that the model's performance was consistent across different subsets of data, reinforcing its generalizability. The minimal variation between folds increased my confidence in the model's stability.

- In a **high-stakes domain** like cancer detection, how would you handle **false positives vs. false negatives**?

In cancer detection, false negatives (missing a cancer diagnosis) are particularly dangerous because they delay treatment and worsen outcomes. However, false positives can cause unnecessary anxiety and lead to invasive follow-up procedures. To address this:

- **Threshold Adjustment & Cost-Sensitive Learning:** I would adjust the decision threshold to favor sensitivity (reducing false negatives) while still keeping specificity at an acceptable level. Incorporating a cost matrix helps assign a higher penalty to false negatives.
- **Secondary Confirmatory Testing:** For borderline cases, implementing an additional diagnostic test could help validate the initial prediction.
- **Decision Curve Analysis:** This method can help determine the net clinical benefit at different threshold levels, ensuring the chosen balance aligns with patient safety and healthcare resource optimization.

5b. Salt-Pepper Noise Reduction (3rd SEM Minor Project)

1. Image Processing Basics:

- You implemented **average and median filters** in both Python and C. Can you explain the **difference** between these filters and why median filtering is typically effective for salt-and-pepper noise?

Average vs. Median Filters

Average Filter:

This is a linear filter that computes the mean of pixel values within a defined window. While it smooths the image by reducing random noise, it tends to blur edges and fine details, and it's not robust against extreme pixel values.

Median Filter:

A non-linear filter that replaces each pixel with the median value of its neighborhood. It's particularly effective for salt-and-pepper noise because such noise manifests as extreme values (either very high or low). The median operation inherently discards these outliers while preserving edges and details, resulting in a cleaner image.

- How did you **measure** the performance or effectiveness of your noise reduction?

I evaluated the effectiveness of the noise reduction using both quantitative and qualitative methods:

2. **Quantitative Metrics:** I computed the Mean Squared Error (MSE) and the Peak Signal-to-Noise Ratio (PSNR) between the original and filtered images. Lower MSE and higher PSNR indicate better noise reduction performance.
3. **Visual Inspection:** I compared the output images to visually assess the preservation of important image details while removing the salt-and-pepper noise.

4. Implementation & Web Deployment:

- You built a **web application** using **Django**, **Tailwind CSS**, and **OpenCV**. How did you integrate OpenCV in a Django environment for real-time image processing?

For the web application, I used the **opencv-python** package. In Django:

5. I created a view that handles image uploads from the user.
6. Once an image is received, the view uses OpenCV functions to apply the selected filtering (average or median) in real time.
7. The processed image is then encoded (e.g., to JPEG) and sent back to the front end as part of the HTTP response.

8. Tailwind CSS was used to design a responsive and modern UI for the application.

- Did you face any **efficiency** or **scalability** issues when processing larger images or multiple requests simultaneously?

Processing larger images and handling multiple simultaneous requests posed some challenges:

- **CPU Load:** Real-time image processing is CPU intensive, so larger images took more time to process.
 - **Concurrent Requests:** To address scalability, I considered using asynchronous request handling (e.g., via Django channels or Celery for offloading processing tasks) and caching processed images when appropriate.
 - **Optimization Strategies:** Techniques like image resizing, optimizing the filtering code in C extensions, and possibly deploying the application with a WSGI server (such as Gunicorn with multiple workers) helped mitigate these issues.
-

6. Technical Skill Deep-Dive

1. Data Structures & Algorithms:

- Given your coding background, how do you approach **time complexity** vs. **space complexity** trade-offs?

I start by analyzing the problem requirements and constraints, then select an algorithm that offers an acceptable balance between time complexity and space usage. For instance:

2. **When speed is critical**, I might choose an algorithm with lower time complexity even if it uses extra space (e.g., dynamic programming with memoization).
3. **When memory is a limitation**, I might opt for an in-place algorithm even if it has a slightly higher time cost.

I usually analyze these trade-offs using Big-O notation and by considering worst-case and average-case scenarios.

- Can you describe a scenario where you would use a **priority queue** vs. a **hash map**?

If you're implementing a scheduler (or Dijkstra's algorithm), a priority queue lets you always access the next highest-priority task or shortest path candidate. In contrast, if you're maintaining a cache of previously computed results (like memoization in DP), a hash map is the appropriate choice.

4. OOP & Design Principles:

- You've studied **OOP** in C++ and have used Python/JavaScript. How do you decide when to use **inheritance** vs. **composition**?

Inheritance is best used when there is a clear "is-a" relationship. For example, if you have a class hierarchy where a *Car* is a type of *Vehicle*, you might inherit common behavior in a base class.

Composition ("has-a" relationship) is preferred for flexibility and to avoid tight coupling. For instance, in one project I built a chatbot where instead of creating deep inheritance hierarchies for different response strategies, I composed the chatbot from several independent modules (e.g., natural language processing, intent recognition, response generation) which could be easily swapped or extended.

- What are some **common design patterns** you find useful, and can you give a real-world example from one of your projects?

Some of the design patterns I frequently use include:

5. **Singleton Pattern:** Ensuring a class (such as a configuration manager) has only one instance.
 6. **Factory Pattern:** Abstracting the creation of objects, for example, dynamically instantiating different types of filters or algorithms based on user input.
 7. **Strategy Pattern:** Enabling interchangeable algorithms. In my chatbot project, I implemented the Strategy pattern to switch between different response-generation strategies (like rule-based vs. AI-driven) without changing the chatbot's overall structure.
 8. **Observer Pattern:** Useful in web apps for event handling, for example, updating UI components when data changes.
- These patterns improve modularity, testability, and maintainability.

9. Database & SQL:

- What's your experience with **SQL queries**? Could you walk me through a **complex query** you've written or a **schema design** for one of your projects?

I have designed and optimized several relational schemas. For instance, in one project (e.g., a conversation logging system for a chatbot) I designed a normalized schema with tables for users, sessions, and messages. I used foreign keys to maintain referential integrity and indexing on frequently queried fields to speed up searches.

Example Query:

I once wrote a query that joined user data, session logs, and message details using multiple

JOIN clauses, aggregated messages per session with **GROUP BY**, and filtered sessions with a **HAVING** clause based on activity thresholds.

Complexity: This query involved subqueries, window functions, and common table expressions (CTEs) to process hierarchical conversation data.

- If you had to optimize a slow query, what steps would you take?

When facing a slow query, my steps include:

Analyzing the Query Plan: Identify bottlenecks using tools like EXPLAIN or the query profiler.

Indexing: Adding or adjusting indexes on columns involved in joins and where clauses.

Query Refactoring: Simplifying subqueries or breaking them into temporary tables/CTEs, and avoiding unnecessary columns or complex expressions.

Caching: If the query results don't change frequently, implementing caching strategies can significantly reduce load.

10. Operating Systems & Networking:

- You've mentioned **Operating Systems** and **Computer Networks** in your coursework. Can you describe how **process scheduling** works or name a few scheduling algorithms?

Process scheduling is the mechanism by which an operating system decides the order and duration of process execution on the CPU. Common scheduling algorithms include:

11. **First-Come-First-Serve (FCFS):** Simple and fair, but can lead to the "convoy effect."

12. **Shortest Job First (SJF):** Minimizes average waiting time but may cause starvation of longer processes.

13. **Round Robin (RR):** Provides fair time slicing for all processes and is commonly used in time-sharing systems.

14. **Priority Scheduling:** Processes are scheduled based on priority, which can be static or dynamic.

These algorithms balance throughput, response time, and fairness.

- How does a **TCP handshake** differ from a **UDP** transaction, and when might you choose one protocol over the other?

TCP:

- **Connection-oriented:** Establishes a connection via a three-way handshake (SYN, SYN-ACK, ACK).
- **Reliability:** Ensures data is delivered, in order, and retransmits lost packets.
- **Use Case:** Chosen when data integrity is crucial (e.g., file transfers, web browsing).

UDP:

15. **Connectionless:** No handshake; packets are sent without establishing a connection.
16. **Low Overhead:** Faster but doesn't guarantee delivery or order.
17. **Use Case:** Ideal for real-time applications like streaming or online gaming where low latency is more critical than perfect accuracy.

18. Version Control & Collaboration:

- You list **Git/GitHub** in your skills. How do you handle **branching** and **merging** in a multi-developer environment?
- Can you describe a time you resolved a **merge conflict**?

Handling Branching and Merging in a Multi-Developer Environment:

I follow a structured branching strategy—typically the “Git Flow” or “feature branch” model—to ensure smooth collaboration. Each new feature or bug fix is developed in its own branch, separate from the main (often called *develop* or *master*) branch. Developers open pull requests to merge changes back into the main branch, where code reviews and automated tests are performed before integration. This approach minimizes conflicts and ensures that the main branch remains stable. Additionally, I often use rebasing to keep my feature branch updated with the latest changes from the main branch, reducing the likelihood of merge conflicts when it's time to merge.

Resolving Merge Conflicts – A Real-World Example:

There was a time when two team members and I were working on overlapping modules in a web application. We encountered a merge conflict when merging our feature branches into the *develop* branch. The conflict arose from simultaneous changes in the same function within a shared file. I resolved the conflict by:

1. Using Git's diff and merge tools (such as the VSCode Git integration and Beyond Compare) to compare the conflicting sections.
2. Discussing with my teammates to understand the intent behind each change.
3. Manually merging the code to integrate the best aspects of both changes while ensuring that functionality and performance were maintained.
4. Running our automated tests and performing manual QA to verify that the resolved code worked as expected.

This collaborative and systematic approach ensured that our codebase remained consistent and that no critical functionality was lost during the merge.

7. Behavioral & Teamwork

1. Team Projects:

- Could you share an instance where you faced a **technical disagreement** with a teammate? How did you resolve it?

In one project, my team was building a web application, and a teammate and I disagreed on whether to use a NoSQL or SQL database. I argued for a relational database due to its robust transaction support, while my teammate favored NoSQL for its scalability. We resolved this by organizing a meeting where we listed the pros and cons, ran a small proof-of-concept for both approaches, and consulted our project requirements. Ultimately, we opted for a hybrid model: using a SQL database for critical transactional data and a NoSQL solution as a caching layer. This experience taught me that data-driven decisions and open communication are key to resolving technical disagreements.

- How do you **prioritize tasks** when juggling multiple projects or deadlines?

I use task management tools like Trello to break projects into smaller tasks with clear deadlines. I prioritize based on impact and urgency, often employing agile methods such as daily stand-ups to adjust priorities on the fly. This structured approach helps me stay organized and ensures that critical deliverables are met even when juggling multiple projects.

2. Challenges & Failures:

- Tell me about a **setback or bug** you encountered in one of your projects. How did you troubleshoot and fix it?

I once encountered a persistent bug in a feature that caused unexpected failures with certain edge-case inputs. Standard debugging wasn't enough, so I enhanced logging and wrote extensive unit tests to narrow down the problem. I eventually discovered the bug was triggered by an unexpected null value and fixed it by adding proper input validation. This experience reinforced the importance of systematic troubleshooting, comprehensive testing, and anticipating edge cases.

- What did you **learn** from that experience?

I led a small team during a hackathon where we had only 48 hours to build a prototype. I delegated tasks based on each member's strengths—front-end, back-end, and UI design—and set clear milestones with regular check-ins via Slack. This ensured that everyone was aligned, and we successfully delivered a working prototype on time.

3. Leadership & Initiative:

- Have you ever **led a small team** or a coding group? How did you **delegate tasks** and keep everyone aligned?
-

8. Fit for Wells Fargo & Future Goals

1. Relevance to Financial Services:

- How do you see your **web development** and **data analysis** skills applying to **banking/finance** solutions at Wells Fargo?
- Are you familiar with **risk analysis** or **compliance** considerations in software design?

My web development and data analysis skills can directly support Wells Fargo's initiatives in developing secure and user-friendly banking solutions. For instance, my experience with Django and real-time data processing can be applied to build dynamic financial dashboards, while my data analysis expertise can help in developing models for risk management and fraud detection. I'm also aware of risk analysis and compliance considerations—key factors when designing software for a regulated industry like finance.

2. Growth & Aspirations:

- Where do you see yourself **3–5 years** from now? Which technologies or domains are you most passionate about exploring further?
- What do you **hope to learn** from this internship, and how can we support your development?

In the next 3–5 years, I see myself evolving into a role that blends technical expertise with strategic decision-making, perhaps as a technical lead or a solutions architect in the financial domain. I'm particularly passionate about exploring technologies like blockchain, machine learning for fraud detection, and cloud computing to enhance financial services. Through this internship, I hope to deepen my understanding of financial systems, learn about industry-specific compliance and risk management practices, and gain mentorship that will help me drive innovation at Wells Fargo.
